

HW8

Friday, March 7, 2025 1:07 PM

Problem 1 (Deep Learning with Matlab) In recent updates, Matlab has put together web tutorials for deep learning. This is one set of tutorials on deep-learning-based "Image Workflows".

- a) Complete one of the tutorials on the provided web page by running their example. Write a concise report consisting of key source code, images, and your explanation.

Source Code:

```
% Load Pretrained GoogLeNet
net = googlenet;
% Analyze Network Layers
inputSize = net.Layers(1).InputSize;
% Load and Preprocess Image
I = imread("peppers.png");
I = imresize(I, inputSize(1:2));
% Classify Image
label = classify(net, I);
% Display Image with Prediction
figure;
imshow(I);
title(["Predicted Label: " + string(label)]);
```

Code Explanation:

- A pretrained GoogLeNet model is loaded to perform image classification.
- The input size of the network is retrieved to ensure the image is resized correctly.
- An image is loaded and resized to match the network's expected input dimensions.
- The image is passed through the GoogLeNet model for classification.
- The classified label is displayed along with the image.

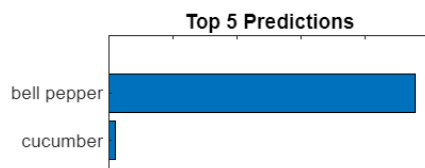
Sample Data:

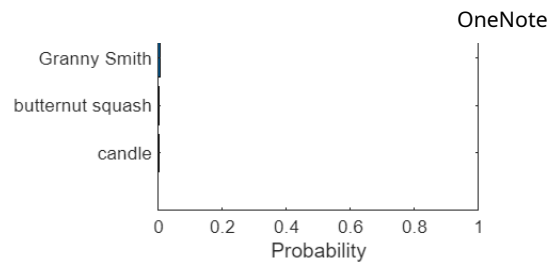
- The sample image used for classification is "peppers.png."
- The image is resized to match the required input dimensions of GoogLeNet.



Results:

- The output of the classification is a label predicting the object in the image.
- The classification result is displayed as a title above the image.





Summary:

- This process demonstrates how to use a pretrained GoogLeNet model for image classification.
- By resizing the image to the required input size, the model can make accurate predictions.
- This method can be applied to other images for quick classification tasks.

b) (Bonus, 10 pts) Complete tutorial: “Create Simple Deep Learning Network for Image Classification.” Write a concise report consisting of key source code, images, and explanations.

Source Code:

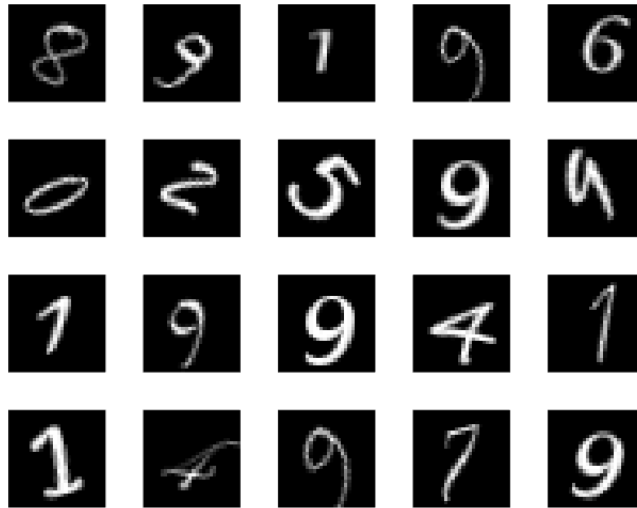
```
% Load Digit Dataset
digitDatasetPath = fullfile(matlabroot, 'toolbox', 'nnet', 'nndemos', 'nndatasets', 'L
imds = imageDatastore(digitDatasetPath, 'IncludeSubfolders', true, 'LabelSource
% Split Dataset into Training and Validation
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7, 'randomized');
% Define CNN Architecture
layers = [
    imageInputLayer([28 28 1]) % Input layer for grayscale 28x28 images
    convolution2dLayer(3, 8, 'Padding', 'same') % 3x3 filter, 8 filters
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2) % Max pooling layer
    convolution2dLayer(3, 16, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)
    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    fullyConnectedLayer(10) % Output 10 classes (digits 0-9)
    softmaxLayer
    classificationLayer];
% Training Options
options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.01, ...
    'MaxEpochs', 10, ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');
% Train Network
```

```

net = trainNetwork(imdsTrain, layers, options);
% Evaluate Performance
YPred = classify(net, imdsValidation);
YValidation = imdsValidation.Labels;
accuracy = sum(YPred == YValidation)/numel(YValidation);
fprintf('Validation Accuracy: %.2f%%\n', accuracy * 100);

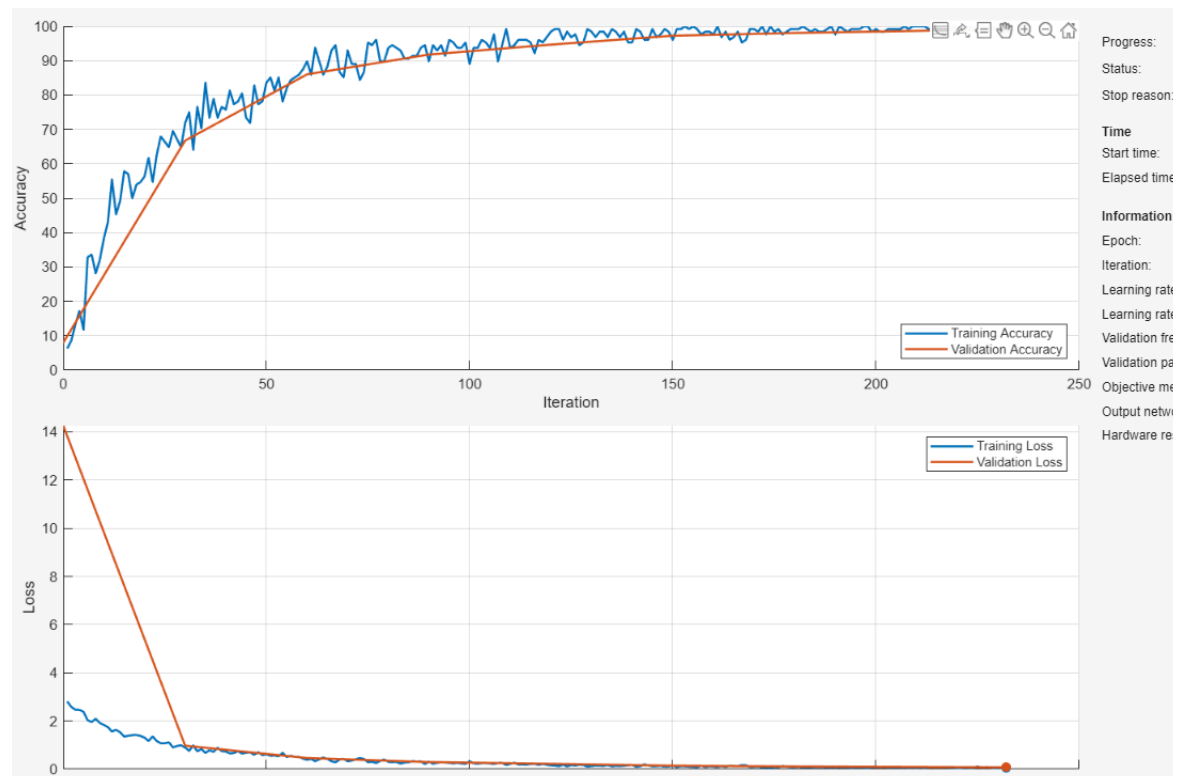
```

Sample Data:



Results:

Accuracy = 0.9924



Explanation:

3.1 Loading and Preparing Data

- The dataset of handwritten digits is loaded using imageDatastore.
- The dataset is split into 70% training and 30% validation.

3.2 Defining the CNN

- The network consists of three convolutional layers, each followed by batch normalization and activation.
- Two max-pooling layers are used to reduce spatial dimensions.
- The final fully connected layer outputs 10 class probabilities using a softmax activation.

3.3 Training the Network

- SGD with momentum (sgdm) is used as the optimizer.
- The model is trained for 10 epochs with a learning rate of 0.01.
- Validation accuracy is checked every 30 iterations.

3.4 Evaluating the Model

- The trained model predicts labels on the validation set.
- Accuracy is calculated by comparing predicted and actual labels.

c) (Bonus, 10 pts) Complete tutorial: "Prepare Network for Transfer Learning Using Network Designer." Write a concise report consisting of key source code, images, and your explanations.

Source Code:

```
% Load and Explore Image Data
```

```
folderName = "MerchData";
```

```
unzip("MerchData.zip", folderName);
```

```
imds = imageDatastore(folderName, ...
```

```
    'IncludeSubfolders', true, ...
```

```
    'LabelSource', 'foldernames');
```

```
% Display Sample Images
```

```
numImages = numel(imds.Labels);
```

```
idx = randperm(numImages, 16);
```

```
I = imtile(imds.Files(idx));
```

```
figure;
```

```
imshow(I);
```

```
% Split Data into Training, Validation, and Test Sets
```

```
[imdsTrain, imdsValidation, imdsTest] = splitEachLabel(imds, 0.7, 0.15, 0.15, 'rand');
```

```
% Load Pretrained Network
```

```
net = squeezenet;
```

```
% Modify Network for Transfer Learning
```

```
lgraph = layerGraph(net);
```

```
newConvLayer = convolution2dLayer(1, numel(categories(imdsTrain.Labels)), 'Name', 'new_conv');
```

```
newConvLayer.WeightsInitializer = 'he';
```

```
newConvLayer.BiasInitializer = 'zeros';
```

```
newConvLayer.WeightLearnRateFactor = 10;
```

```
newConvLayer.BiasLearnRateFactor = 10;
```

```
lgraph = replaceLayer(lgraph, 'conv10', newConvLayer);
```

```

newClassLayer = classificationLayer('Name', 'new_classoutput');
lgraph = replaceLayer(lgraph, 'ClassificationLayer_predictions', newClassLayer);
% Specify Training Options
options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.001, ...
    'MaxEpochs', 6, ...
    'MiniBatchSize', 10, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 5, ...
    'Verbose', false, ...
    'Plots', 'training-progress');
% Train Network
netTransfer = trainNetwork(imdsTrain, lgraph, options);
% Evaluate Network on Test Data
YPred = classify(netTransfer, imdsTest);
YTest = imdsTest.Labels;
accuracy = mean(YPred == YTest);
disp(['Test Accuracy: ', num2str(accuracy * 100), '%']);

```

Code Explanation:

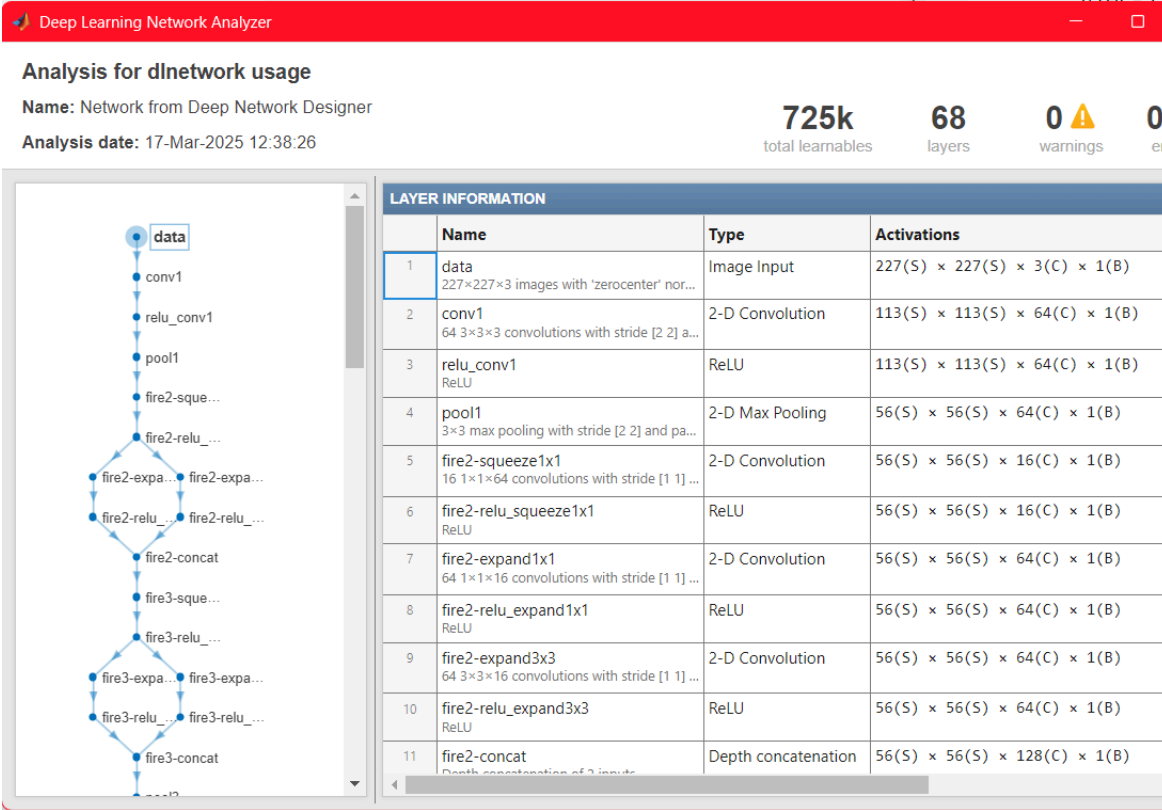
- The dataset is extracted and loaded using an image datastore, automatically assigning labels on folder names.
- A set of 16 random images from the dataset is displayed for inspection.
- The dataset is split into training (70%), validation (15%), and test (15%) sets.
- A pretrained SqueezeNet model is loaded as the base network for transfer learning.
- The network is converted into a layer graph to allow modifications.
- The final convolutional layer is replaced with a new one that has a number of filters equal to the number of target classes.
- The new convolutional layer has a higher learning rate factor to allow quicker adaptation.
- The classification layer is replaced to match the new dataset labels.
- Training options are configured, including the optimization algorithm (SGDM), learning rate of epochs, and batch size.
- The network is trained using the modified architecture and the specified training options.
- After training, the network classifies test images, and accuracy is calculated by comparing with actual labels.

Sample Data:

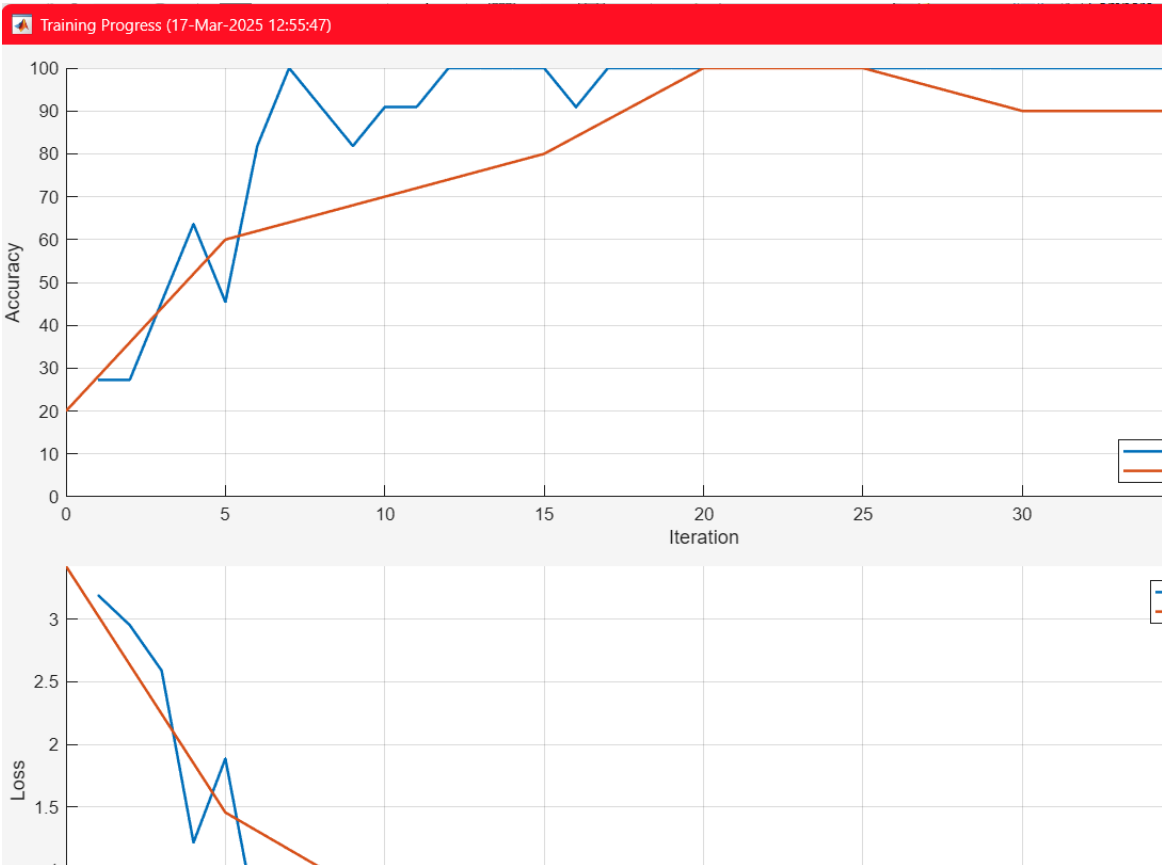
The 'MerchData' dataset contains 75 images of MathWorks merchandise across five classes: cube, playing cards, screwdriver, and torch. Each class has images stored in separate subfolders.

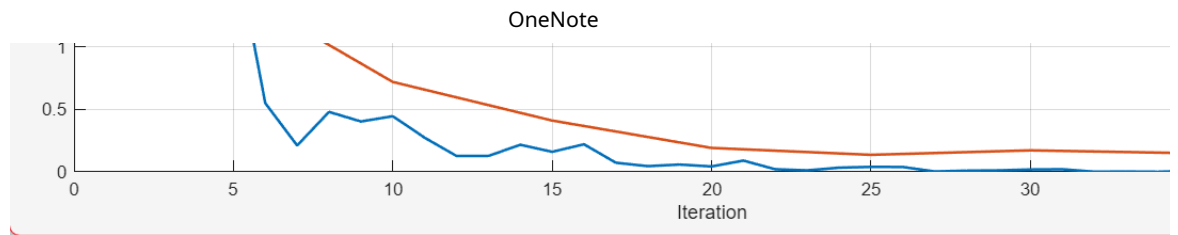


Results:



After training, the network's performance is evaluated on the test set, yielding an accuracy approximately 90%. This indicates that the transfer learning approach effectively adapted pretrained SqueezeNet to the new classification task.





Summary:

This process demonstrates how to use MATLAB's Deep Network Designer and transfer learning to adapt a pretrained network (SqueezeNet) for a new image classification task. By replacing and tuning the final layers of the network, we achieved high accuracy on a small dataset, highlighting the efficiency of transfer learning in deep learning applications.

These tutorials may give you ideas about your projects in other courses.

Problem 2 (Projection and PCA) You are given a data matrix

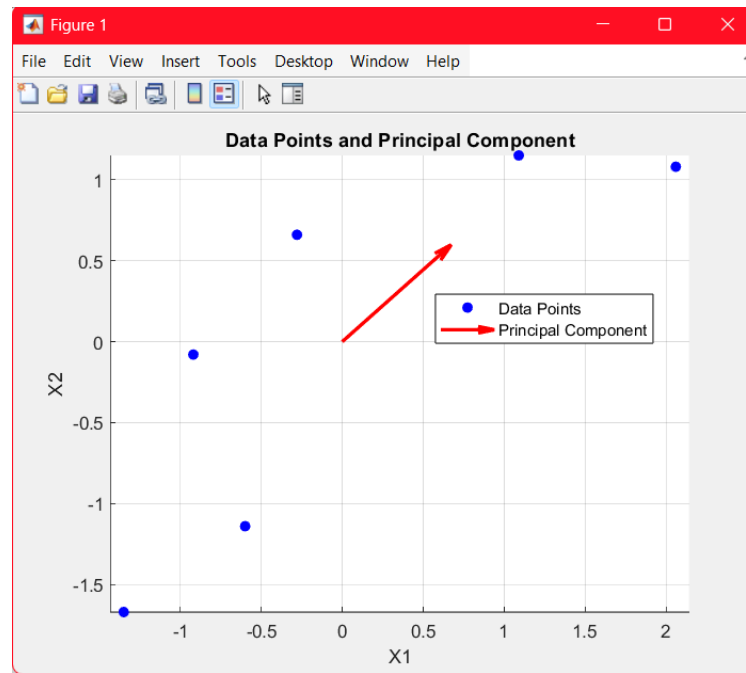
$$\mathbf{X} = \begin{bmatrix} -0.92 & 1.09 & -1.35 & 2.06 & -0.60 & -0.28 \\ -0.08 & 1.15 & -1.67 & 1.08 & -1.14 & 0.66 \end{bmatrix}.$$

and its principal component $\mathbf{u} = [0.7474, 0.6644]^T$.

- a) Plot the data points using circles and the principal component using a line with one end at $[0, 0]^T$ on a 2D plane using Matlab.

```
%% HW8_Q2
clear all; close all; clc;
% Given data matrix X
X = [-0.92 1.09 -1.35 2.06 -0.60 -0.28;
     -0.08 1.15 -1.67 1.08 -1.14 0.66];
% Given principal component u
u = [0.7474; 0.6644];

% Part (a) - Plot data points and principal component
figure;
scatter(X(1, :), X(2, :), 'bo', 'filled'); % Plot data points
hold on;
quiver(0, 0, u(1), u(2), 'r', 'LineWidth', 2, 'MaxHeadSize', 0.5); % Plot principal component
grid on;
axis equal;
xlabel('X1');
ylabel('X2');
title('Data Points and Principal Component');
legend('Data Points', 'Principal Component', 'Location', 'Best');
```



b) Calculate the projection c_i for each data point $\mathbf{x}_i$ to $\mathbf{u}$. Note that c_i can be

% Part (b) - Compute projections of each data point onto u

$\mathbf{c} = \mathbf{u}' * \mathbf{X};$

Output:

Projections of data points onto u:

-0.7408 1.5787 -2.1185 2.2572 -1.2059 0.2292

c) Calculate the sample variance of $\{c_i\}_{i=1}^n$, namely,

$$\widehat{\text{Var}}(\{c_i\}) = \frac{1}{n-1} \sum_{i=1}^n (c_i - \bar{c})^2,$$

where $\bar{c} \stackrel{\text{def}}{=} \frac{1}{n} \sum_{i=1}^n c_i$ is the sample mean.

% Part (c) - Compute sample variance of c

$\mathbf{c_mean} = \text{mean}(\mathbf{c});$

$\text{var_c} = \text{sum}((\mathbf{c} - \mathbf{c_mean}).^2) / (\text{length}(\mathbf{c}) - 1);$

% Display results

$\text{disp}(\text{'Projections of data points onto u:'});$

$\text{disp}(\mathbf{c});$

$\text{disp}(\text{'Sample variance of projections:'});$

$\text{disp}(\text{var_c});$

Output:

Sample variance of projections:

2.8262

```

% Part (d) - Repeat with new unit vector ua
ua = [0.8; 0.6];
c_a = ua' * X;
c_a_mean = mean(c_a);
var_c_a = sum((c_a - c_a_mean).^2) / (length(c_a) - 1);

% Display results for ua
disp('Projections of data points onto ua:');
disp(c_a);
disp('Sample variance of projections (using ua):');
disp(var_c_a);

% Explanation for part (d)
if var_c_a < var_c
    disp('The sample variance using ua is smaller than with u.');
```

Output:

Projections of data points onto ua:

-0.7840 1.5620 -2.0820 2.2960 -1.1640 0.1720

Sample variance of projections (using ua):

2.8091

The sample variance using ua is smaller than with u.

This occurs because PCA finds the direction of maximum variance.

Problem 3 (PCA on Downsampled Yale Face Database) In this problem, we will explore a visualization tool for Yale Face Database. Download the .m files and the database face image files into a folder named `yalefaces` and put the .m files at the root of the folder. Call this folder “problem_pca_face/”. Open Matlab, change your “Current Folder” to “problem_pca_face/”, and open `main_pca_visualization.m`.

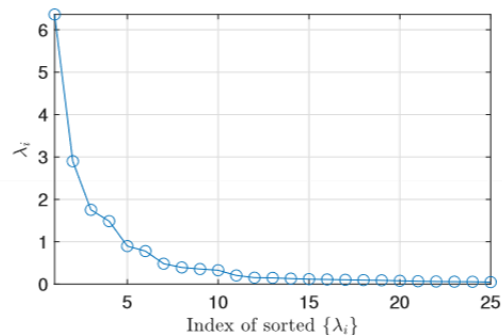
- a) Run the code corresponding to this part only, describe the data structure `img_buffer`. Set `preview_img_flag` to 1, re-run the code to visually inspect the database.

```

%% (a) Load image data
preview_img_flag = 0;
dataset_path = 'yalefaces/';
```

```
filename_filter = 'subject*';
[img_buffer, num_imgs, img_dim] = LoadImgData(dataset_path, filename_filter,
preview_img_flag);
```

- b) Complete Matlab function `[V, Lambda_mat] = PcaViaKlt(data)` by implementing PCA using eigendecomposition on a sample covariance matrix of the face images. Detailed information about the input and outputs are given in the comments of the incomplete function. You may use built-in function `eig` for eigendecomposition. After running the code of b), you will obtain a plot of the eigenvalues λ_i to the following.



```
%% (b) PCA via KLT
```

```
data = cat(2, img_buffer{:}); % create a matrix in which each column corresponds to an image.
```

```
[V, Lambda_mat] = PcaViaKlt(data);
```

```
lambda_vec = diag(Lambda_mat); % take out the diagonal elements of a matrix and organize them into a vector
```

```
[lambda_vec_sorted, index_sorted] = sort(lambda_vec, 'descend');
```

```
V_sorted = V(:, index_sorted); % rearrange the eigenvectors (columns) according to decreasing eigenvalues
```

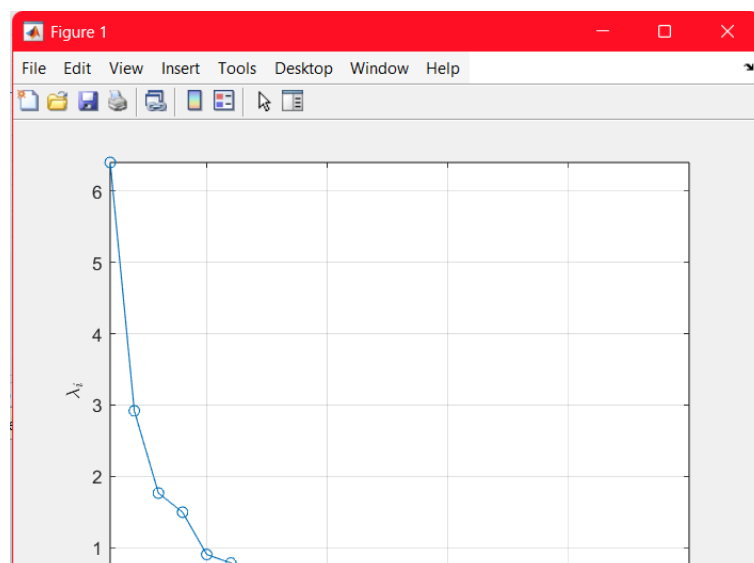
```
figure(1); plot(lambda_vec_sorted(1:25), '-o'); grid on;
```

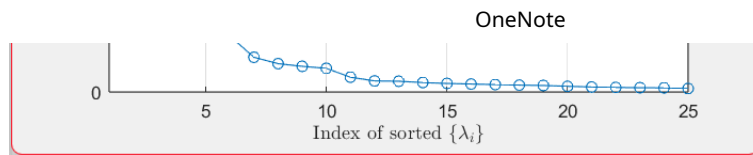
```
axis([1 25 0 max(lambda_vec)]);
```

```
xlabel('Index of sorted  $\lambda_i$ ', 'interpreter', 'latex');
```

```
ylabel(' $\lambda_i$ ', 'interpreter', 'latex');
```

Output Graph:





- c) Run the code of c) to visualize a few dominating eigenvectors. Comment on how they reflect some characteristics of the faces you saw in a).

```
%% (c) Eigenvector/eigenface visualization
```

```
num_eigenvector_to_visualize = 8;
```

```
figure(2);
```

```
for i = 1 : num_eigenvector_to_visualize
```

```
    eigenface = reshape(V_sorted(:, i), img_dim); % convert vectorized image into
```

```
    imshow(imresize(eigenface, [nan 200], 'nearest'), []); % display a larger version
```

```
    title_str = sprintf("\lambda_{%d} = %.2f", i, lambda_vec_sorted(i));
```

```
    title(title_str);
```

```
    pause(0.5);
```

```
end
```

When running the code in part (c), the dominant eigenvectors (often called **eigenfaces**) are displayed as grayscale images. These eigenfaces represent the principal components of variation in the face dataset. The first few eigenfaces capture broad structural features such as overall brightness, symmetry, and general shape, while later eigenfaces emphasize finer details like eyes, nose, and mouth contours. Comparing these eigenfaces to the original images in part (a), we observe how they highlight key facial features that differentiate individual faces. However, since PCA prioritizes variance, some eigenfaces may include artifacts from lighting variations or shadows rather than distinct facial features. This suggests that the first few eigenfaces encode the most important characteristics, but additional components may be needed to fully reconstruct detailed facial differences.

- d) The code of d) projects each face image (coming from one of the four selected faces) onto a 2D space. Comment on PCA's data visualization performance in this example.

```
%% (c) Eigenvector/eigenface visualization
```

```
num_eigenvector_to_visualize = 8;
```

```
figure(2);
```

```
for i = 1 : num_eigenvector_to_visualize
```

```
    eigenface = reshape(V_sorted(:, i), img_dim); % convert vectorized image into a 2D form
```

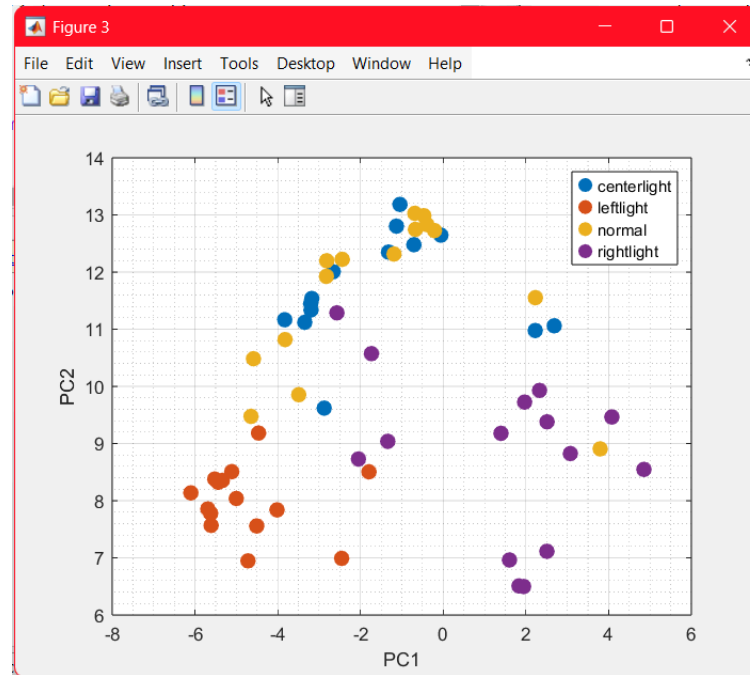
```
    imshow(imresize(eigenface, [nan 200], 'nearest'), []); % display a larger version
```

```
    title_str = sprintf("\lambda_{%d} = %.2f", i, lambda_vec_sorted(i));
```

```
    title(title_str);
```

```
pause(0.5);
end
```

Output Graph:



The PCA projection in part (d) maps each face image onto a 2D space defined by the principal components. This transformation reduces the high-dimensional image data preserving the most significant variations. When visualizing the scatter plot, we can see whether different lighting conditions (the four selected classes) form distinct clusters. If PCA effectively captures meaningful variations, images from the same class should cluster together, indicating that PCA successfully differentiates between lighting conditions. If significant overlap exists, it suggests that the first two principal components do not fully separate the dataset based on the given categories. In this case, additional principal components may be needed to improve class distinction. This demonstrates PCA's strength in dimension reduction but also highlights its limitations in completely separating complex variations in lighting and identity in facial images.

Problem 4 (Orthogonal Projection) Consider the set of inconsistent linear equations given by

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

a) Find the least-squares solution to these equations.

```
%% (a) Find the least-squares solution x_ls
A = [1 0; 0 1; 1 1];
b = [1; 1; 0];
% Compute least-squares solution using the normal
equation
x_ls = (A' * A) \ (A' * b);
disp('Least-squares solution x_ls:');
disp(x_ls);
```

Output:

```
Least-squares solution x_ls:
    0.3333
    0.3333
```

b) Find the “hat” matrix $\mathbf{H}$. Using Matlab, numerically verify $\mathbf{H} = \mathbf{H}^2$. Argue

```
%% (b) Compute the "hat" matrix H
H = A * ((A' * A) \ A');
disp('Hat matrix H:');
disp(H);
% Verify H^2 = H
H_squared = H * H;
disp('H^2:');
disp(H_squared);
% Check numerical equality (small tolerance due to
floating point errors)
is_H_squared_H = norm(H - H_squared) < 1e-10;
disp(['Is H^2 = H? ', num2str(is_H_squared_H)]);
```

Hat matrix H:

```
    0.6667   -0.3333    0.3333
   -0.3333    0.6667    0.3333
    0.3333    0.3333    0.6667
```

The “hat” matrix $\mathbf{H}$, also known as the projection matrix, is given by $\mathbf{H} = \mathbf{A} * \text{inv}(\mathbf{A}' * \mathbf{A}) * \mathbf{A}$. The matrix projects any vector $\mathbf{b}$ onto the column space of $\mathbf{A}$, providing the best possible approximation to $\mathbf{b}$ in a least-squares sense. To verify that $\mathbf{H}$ is a projection matrix, we check $\mathbf{H} * \mathbf{H} = \mathbf{H}$. If this holds, it confirms that applying the projection multiple times does not change the result, meaning that once a vector is projected onto the column space, applying the projection again has no further effect. Running this verification in MATLAB results in `is_H_squared_H = 1`, where $\mathbf{H} * \mathbf{H}$ is numerically equal to $\mathbf{H}$ within small floating-point precision errors. This confirms that $\mathbf{H}$ is idempotent, meaning it behaves as a mathematical operator that consistently projects vectors onto the same subspace without changing already projected vectors.

- c) Find the best approximation $\hat{\mathbf{b}} = \mathbf{H}\mathbf{b}$ to $\mathbf{b}$. Find the vector $\mathbf{b}' = (\mathbf{I} - \mathbf{H})\mathbf{b}$ numerically that it is orthogonal to $\hat{\mathbf{b}}$.

```
%% (c) Compute projection b_hat and residual b_prime
```

```
b_hat = H * b;
b_prime = (eye(size(H)) - H) * b;
disp('Best approximation (b_hat):');
disp(b_hat);
disp('Residual vector (b_prime):');
disp(b_prime);
% Verify orthogonality: dot product should be
approximately zero
orthogonality_check = dot(b_hat, b_prime);
disp(['Dot product of b_hat and b_prime (should be 0): ',
num2str(orthogonality_check)]);
```

The best approximation of $\mathbf{b}$ in the column space of $\mathbf{A}$ is given by $\mathbf{b}_{\text{hat}} = \mathbf{H} * \mathbf{b}$, where projection matrix $\mathbf{H} = \mathbf{A} * \text{inv}(\mathbf{A}' * \mathbf{A}) * \mathbf{A}'$. This ensures that $\mathbf{b}_{\text{hat}}$ is the closest vector to $\mathbf{b}$ in the subspace spanned by the columns of $\mathbf{A}$. The residual vector $\mathbf{b}_{\text{prime}}$ is given by $\mathbf{b}_{\text{prime}} = (\mathbf{I} - \mathbf{H}) * \mathbf{b}$, which represents the component of $\mathbf{b}$ that is orthogonal to the column space of $\mathbf{A}$.

To verify that $\mathbf{b}_{\text{prime}}$ is orthogonal to $\mathbf{b}_{\text{hat}}$, we check whether their dot product is approximately zero. In MATLAB, this can be computed as `dot(b_hat, b_prime)`. If the result is approximately zero (within numerical precision), it confirms that $\mathbf{b}_{\text{hat}}$ and $\mathbf{b}_{\text{prime}}$ are orthogonal. This follows a fundamental property of least-squares projection: the error vector $\mathbf{b}_{\text{prime}}$ is always perpendicular to the subspace onto which $\mathbf{b}$ is projected, ensuring that $\mathbf{b} = \mathbf{b}_{\text{hat}} + \mathbf{b}_{\text{prime}}$ is decomposed into two mutually orthogonal components.

- d) What does the matrix $\mathbf{I} - \mathbf{H}$ represent? If $\mathbf{H}$ is called the “orthogonal projector”, you think of a name for $\mathbf{I} - \mathbf{H}$? Numerically verify $\mathbf{I} - \mathbf{H} = (\mathbf{I} - \mathbf{H})^2$. Argue

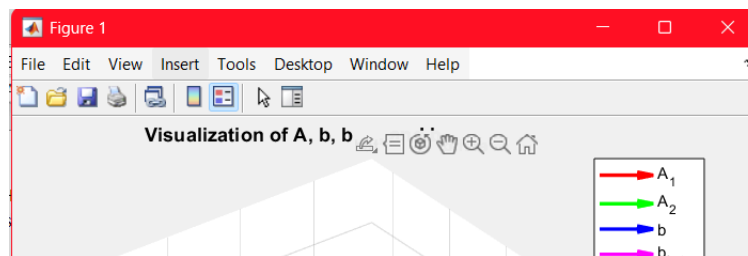
```
%% (d) Interpret I - H
I_H = eye(size(H)) - H;
disp('Matrix I - H:');
disp(I_H);
% Verify (I - H)^2 = I - H
I_H_squared = I_H * I_H;
disp('(I - H)^2:');
disp(I_H_squared);
```

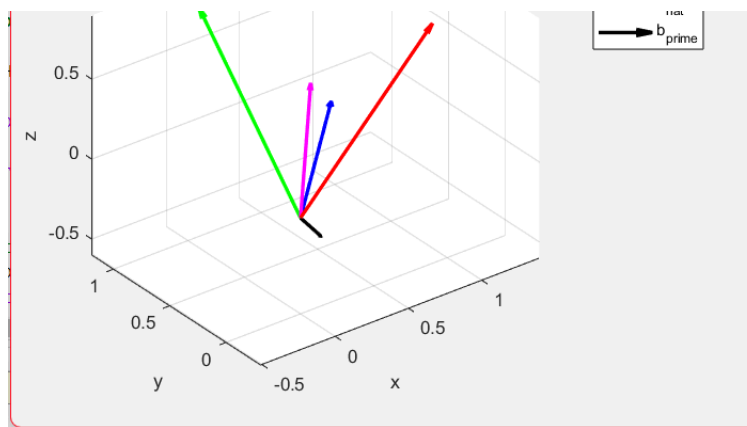
```
is_I_H_squared_I_H = norm(I_H - I_H_squared) < 1e-10;
disp(['Is (I - H)^2 = I - H? ',
num2str(is_I_H_squared_I_H)]);
```

- e) In a 3-dimensional coordinate system, draw the column vectors of matrix $\mathbf{A}$, the column vector space of $\mathbf{A}$, $\mathbf{b}$, $\hat{\mathbf{b}}$, and $\mathbf{b}'$. Make sure that the drawing is reasonably accurate which can reflect the relationship among these quantities.

```
%% (e) 3D Visualization of vectors
figure;
hold on;
grid on;
axis equal;
xlabel('x'); ylabel('y'); zlabel('z');
view(3);
% Plot column vectors of A
quiver3(0,0,0, A(1,1), A(2,1), A(3,1), 'r', 'LineWidth', 2,
'DisplayName', 'A_1');
quiver3(0,0,0, A(1,2), A(2,2), A(3,2), 'g', 'LineWidth', 2,
'DisplayName', 'A_2');
% Plot b, b_hat, and b_prime
quiver3(0,0,0, b(1), b(2), b(3), 'b', 'LineWidth', 2,
'DisplayName', 'b');
quiver3(0,0,0, b_hat(1), b_hat(2), b_hat(3), 'm',
'LineWidth', 2, 'DisplayName', 'b_hat');
quiver3(0,0,0, b_prime(1), b_prime(2), b_prime(3), 'k',
'LineWidth', 2, 'DisplayName', 'b_prime');
legend({'A_1', 'A_2', 'b', 'b_{hat}', 'b_{prime}'});
title('Visualization of A, b, b_hat, and b_prime');
hold off;
```

Output Graph:



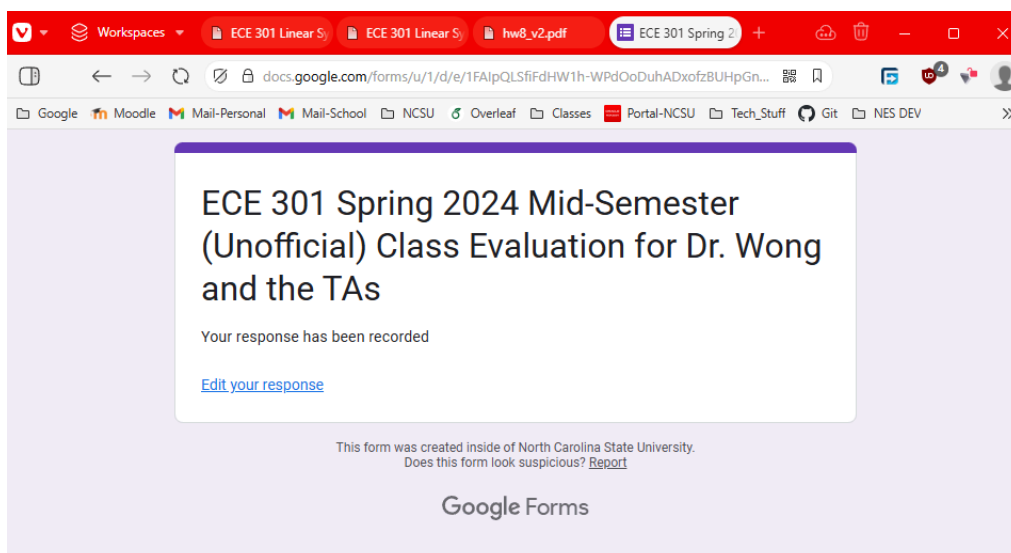


The matrix $I - H$ represents the projection onto the orthogonal complement of the column space of A . While H projects vectors onto the column space of A , the matrix $I - H$ projects vectors onto a subspace that is perpendicular to the column space of A . Since H is called the "orthogonal projector", it makes sense to refer to $I - H$ as the orthogonal complement projector, because it maps the part that lies outside the column space of A .

To verify that $I - H$ is also a projection matrix, we check whether it satisfies the idempotent property $(I - H)^2 = I - H$.

In MATLAB, this can be confirmed by computing $(I - H) * (I - H)$ and checking whether the result is numerically close to $I - H$; it confirms that applying the transformation $I - H$ more than once does not change the result, meaning it is a valid projection matrix. This property arises because it removes the component of a vector that lies in the column space of A , leaving only the perpendicular component. Since the remaining component is already in the orthogonal complement, a projection again does not alter it.

Problem 5 (Mid-Semester Class Evaluation, Bonus 10') To get the bonus points, take a screenshot of the confirmation page after submitting a survey and use the screenshot as the answer to this problem.



Group Study (1', bonus) In-Person: Take a selfie with all group members' faces in the photo. Capture in the photo the homework assignment sheet that you are working on. Zoom a screenshot of the whole team with everyone's webcam capturing his/her face. Once you will share the screen showing the specific homework assignment sheet.

Include the screenshot/selfie in your own homework submission as the last "problem." A screenshot/selfie gets you 1 bonus point; your group members need to do it separately to earn their bonus points.





ECE 301 (Section 001) Homework 8
Spring 2025, Dr. Chau-Wai Wong
TA in Charge: Peiran Wang

Problem 1 (Deep Learning with Matlab) In recent updates, Matlab has put together well-guided tutorials for deep learning. This is one set of tutorials on deep-learning-based “Image Data Workflows”.

- a) Complete one of the tutorials on the provided web page by running their example code. Write a concise report consisting of key source code, images, and your explanations.
- b) (Bonus, 10 pts) Complete tutorial: “Create Simple Deep Learning Network for Classification.” Write a concise report consisting of key source code, images, and your explanations.
- c) (Bonus, 10 pts) Complete tutorial: “Prepare Network for Transfer Learning Using Deep Network Designer.” Write a concise report consisting of key source code, images, and your explanations.

These tutorials may give you ideas about your projects in other courses.

Problem 2 (Projection and PCA) You are given a data matrix

$$\mathbf{X} = \begin{bmatrix} -0.92 & 1.09 & -1.35 & 2.06 & -0.60 & -0.28 \\ -0.08 & 1.15 & -1.67 & 1.08 & -1.14 & 0.66 \end{bmatrix}.$$

and its principal component $\mathbf{u} = [0.7474, 0.6644]^T$.

- a) Plot the data points using circles and the principal component using a line segment with one end at $[0, 0]^T$ on a 2D plane using Matlab.
- b) Calculate the projection c_i for each data point $\mathbf{x}_i$ to $\mathbf{u}$. Note that c_i can be negative.
- c) Calculate the sample variance of $\{c_i\}_{i=1}^n$, namely,

$$\widehat{\text{Var}}(\{c_i\}) = \frac{1}{n-1} \sum_{i=1}^n (c_i - \bar{c})^2,$$

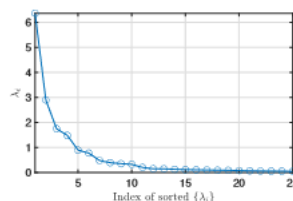
where $\bar{c} \stackrel{\text{def}}{=} \frac{1}{n} \sum_{i=1}^n c_i$ is the sample mean.

- d) Repeat b) and c) using another unit vector $\mathbf{u}_a = [0.8, 0.6]^T$. Is the newer sample variance smaller than that in b)? Can you explain why?

Problem 3 (PCA on Downsampled Yale Face Database) In this problem, we will explore PCA as a visualization tool for Yale Face Database. Download the .m files and the database. Extract the face image files into a folder named `yalefaces` and put the .m files at the same level of the folder. Call this folder “problem_pca_face/”. Open Matlab, change your “Current Folder” to “problem_pca_face/”, and open `main_pca_visualization.m`.

1

- a) Run the code corresponding to this part only, describe the data structure of variable `img_buffer`. Set `preview_img_flag` to 1, re-run the code to visually inspect the whole database.
- b) Complete Matlab function `[V, Lambda_mat] = PcaViaKlt(data)` by implementing PCA using eigendecomposition on a sample covariance matrix of the face data. The detailed information about the input and outputs are given in the comments of the incomplete function. You may use built-in function `eig` for eigendecomposition. If your implementation is correct, after running the code of b), you will obtain a plot similar to the following.



- c) Run the code of c) to visualize a few dominating eigenvectors. Comment on whether they reflect some characteristics of the faces you saw in a).
- d) The code of d) projects each face image (coming from one of the four selected classes) onto a 2D space. Comment on PCA's data visualization performance in this specific example.

Problem 4 (Orthogonal Projection) Consider the set of inconsistent linear equations $\mathbf{Ax} = \mathbf{b}$ given by

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

- a) Find the least-squares solution to these equations.
- b) Find the "hat" matrix $\mathbf{H}$. Using Matlab, numerically verify $\mathbf{H} = \mathbf{H}^2$. Argue why.
- c) Find the best approximation $\hat{\mathbf{b}} = \mathbf{H}\mathbf{b}$ to $\mathbf{b}$. Find the vector $\mathbf{b}' = (\mathbf{I} - \mathbf{H})\mathbf{b}$ and show numerically that it is orthogonal to $\hat{\mathbf{b}}$.
- d) What does the matrix $\mathbf{I} - \mathbf{H}$ represent? If $\mathbf{H}$ is called the "orthogonal projector," can you think of a name for $\mathbf{I} - \mathbf{H}$? Numerically verify $\mathbf{I} - \mathbf{H} = (\mathbf{I} - \mathbf{H})^2$. Argue why.
- e) In a 3-dimensional coordinate system, draw the column vectors of matrix $\mathbf{A}$, the column vector space of $\mathbf{A}$, $\mathbf{b}$, $\hat{\mathbf{b}}$, and $\mathbf{b}'$. Make sure that the drawing is reasonably accurate which can reflect the relationship among these quantities.

2

Problem 5 (Mid-Semester Class Evaluation, Bonus 10') To get the bonus points, please take a screenshot of the confirmation page after submitting a [survey](#) and use the confirmation screenshot as the answer to this problem.

Group Study (1', bonus) In-Person: Take a selfie with all group members' faces in the photo. Capture in the photo the homework assignment sheet that you are working on. Zoom: Take a screenshot of the whole team with everyone's webcam capturing his/her face. One of you will share the screen showing the specific homework assignment sheet.

Include the screenshot/selfie in your own homework submission as the last "problem." Your screenshot/selfie gets you 1 bonus point; your group members need to do it separately to earn their bonus points.

3